

PLAYBYTE

Field	Detail
Source document	Playbyte PRD, Draft v1.0 (11 August 2026), owned by Product Management
Architecture status	v1.2 — supersedes v1.1; adds cold-start crowd context and the guest-to-account soft-prompt mechanism
System owner	Engineering
MVP platforms	Native iOS and Android only
Explicitly excluded at MVP	Web consumer app, real-money wagering, public leaderboard, creator tooling, paid subscription

1 Scope

This document defines the implementation architecture for the Playbyte MVP: the mobile client, application API, PostgreSQL data model, realtime delivery, background processing, editorial CMS, authentication, privacy, sharing and media generation, notifications, observability, deployment, and the scaling path required by the PRD.

It is written for the engineers building the system. Product rationale lives in the PRD; this document assumes that rationale and focuses on how the system is built, why each decision was made over the available alternatives, and what conditions would change it.

2 PRD Requirements → Architecture Traceability

Every PRD requirement with a technical implication maps to a concrete design target below. This table is the contract between product and engineering — if a target changes, this row is where it gets renegotiated.

Area	PRD requirement	Design target
Feed	Full-screen vertical swipe; next 3 cards preload	Client prefetch + cursor-paginated feed API
Response	Captured on selection, not card exit	Immediate POST on tap; idempotent write
Result	Live result visible after response	Authoritative API result + realtime snapshot

Area	PRD requirement	Design target
Realtime	95% of viewers receive update within 2 sec	Aggregated broadcast snapshots, not per-event fanout
Scale	100,000 concurrent connected sessions at MVP	Capacity-tested realtime layer
Scale path	1M+ connected sessions	Replaceable realtime provider behind an interface; horizontal API/workers
Availability	99.9% monthly	Managed DB + stateless API + retry/fallback
Feed latency	No empty/loading state > 500ms in normal conditions	Prefetch + client buffer
App start	Under 3 sec to first interactive moment, mid-tier device	Minimal startup dependency path
Privacy	Response visibility enforced server-side	Backend policy layer + DB-level authorization
Accessibility	WCAG 2.1 AA; tap alternative to swipe	Accessible controls and labels in the client
CMS	Publish in < 2 min; second approval for restricted topics	Admin workflow modeled as an explicit state machine
Sharing	Branded result card, 2 taps, native share sheet	Server-rendered image asset + deep-linkable share URL
Notifications	3/day default cap, per-type opt-out, no stale sends	Push module enforces cap and expiry before dispatch
Privacy & data	GDPR/CCPA, opt-in contact sync, export/delete	Consent ledger, data-subject workflows, retention policy

3 Architecture Decision

The MVP uses a modular monolith with Supabase as the managed data platform.

```

Mobile App (iOS / Android)
|
| HTTPS
v
Playbyte API — NestJS / TypeScript (modular monolith)
|
+---- Supabase PostgreSQL      (system of record)
+---- Supabase Auth            (identity)
+---- Supabase Storage          (avatars, share assets)
+---- Redis / Valkey            (added when measured need appears)
+---- Queue + Worker process    (async side effects)
+---- RealtimePublisher          (interface, not a vendor call)
    |
    +---- Supabase Broadcast      (initial transport)
    +---- AppSync / Ably / custom WS (future, only if required)

```

Why a modular monolith, and why now

Playbyte has strong relational requirements — users, moments, responses, friendships and privacy settings are highly interdependent and benefit from transactional guarantees a single relational database gives for free. It also has one component with a genuinely different scaling shape: realtime fanout to up to 100,000 concurrent viewers. Splitting the system into services before that asymmetry is proven would trade a real, measurable cost (operational overhead for a small team) for a benefit that doesn't exist yet.

The monolith is kept modular specifically so the realtime path can be extracted later without a rewrite: it sits behind a RealtimePublisher interface from day one, so Supabase Broadcast is a swappable implementation detail, not a structural assumption baked into the rest of the codebase.

Decision rule. Default to the boring, already-managed option (Supabase Postgres/Auth/Storage/Broadcast). Introduce new infrastructure only when a measured workload — not a hypothetical one — requires it. See Section 26.

4 Components

Component	Responsibility	Initial choice
Mobile	Feed UI, interaction capture, client cache, realtime subscription	React Native + Expo + TypeScript
API	All product/business rules and authenticated operations	NestJS + Fastify
Database	Users, moments, responses, friendships, privacy, CMS state	Supabase PostgreSQL
Authentication	User sessions, OAuth, email, account lifecycle	Supabase Auth
Object storage	Avatars and generated share-card assets	Supabase Storage
Realtime	Crowd snapshots to active viewers	Supabase Broadcast, behind RealtimePublisher
Cache	Hot counters, rate limits, short-lived data	Redis/Valkey, added when measured need appears
Queue	Async work and retries	Queue abstraction + worker process
Admin / CMS	Moment authoring, scheduling, moderation, approvals	Next.js + TypeScript
Sharing	Branded result-card rendering and deep-link resolution	Server-side image render + Supabase Storage + universal links
Push	Live Now, Trending Pulse, Friend Activity	APNs + FCM
Analytics	Product events	PostHog
Error tracking	Runtime failures	Sentry

5 Backend Module Structure

```
backend/src/
modules/
```

```

auth/  users/  profiles/  interests/
moments/  feed/  responses/  crowd/
friends/  sharing/  notifications/  recap/
moderation/  cms/
infrastructure/
  postgres/  cache/  queue/  realtime/  storage/  push/
common/
  errors/  auth/  validation/  logging/  telemetry/
main.ts

```

Dependencies flow inward. Controllers hold no business rules; domain and application code never call vendor SDKs directly. All vendor-specific code — Supabase client calls, APNs/FCM calls, Redis commands — lives under `infrastructure/` behind an interface owned by the module that needs it. This is what makes the Section 26 migration path (swap Supabase Broadcast for AppSync, add a second cache, replace the queue) a localized change instead of a rewrite.

6 API Design

Method	Endpoint	Purpose	Auth
POST	<code>/v1/guest/sessions</code>	Create guest play session	None
GET	<code>/v1/feed</code>	Fetch next feed page	Guest or user
GET	<code>/v1/moments/:id</code>	Fetch a moment	Guest or user
GET	<code>/v1/moments/:id/result</code>	Authoritative current result	Guest or user
POST	<code>/v1/moments/:id/responses</code>	Submit response	Guest or user
POST	<code>/v1/moments/:id/share-card</code>	Generate branded share-card asset	Guest or user
GET	<code>/v1/me</code>	Current profile	User
PATCH	<code>/v1/me/profile</code>	Update profile	User
PATCH	<code>/v1/me/privacy</code>	Update privacy defaults	User
GET	<code>/v1/me/recap/weekly</code>	Weekly recap	User
POST	<code>/v1/me/push-tokens</code>	Register push token	User
PATCH	<code>/v1/me/notification-preferences</code>	Per-type notification opt-out	User
GET	<code>/v1/friends</code>	List connections	User
POST	<code>/v1/friends/requests</code>	Create connection request	User

The feed uses cursor pagination. Response creation requires an `Idempotency-Key` header. Every error response returns a consistent body with an application error code and a request ID, so a support ticket or log line can be traced to the exact request.

7 Database Model

Entity	Key fields	Constraints / indexes
users	id, auth_subject, status, created_at	Unique auth_subject
guest_sessions	id, token_hash, moments_responded_count, expires_at, converted_user_id	Unique token hash; expiry index
profiles	user_id, display_name, avatar_key, bio, default_visibility	1:1 with users
categories	id, slug, name	Unique slug
user_interests	user_id, category_id	Unique pair
moments	id, type, category_id, status, starts_at, ends_at, prompt	Status/time/category indexes
moment_options	id, moment_id, label, sort_order	Unique (moment_id, sort_order)
responses	id, moment_id, user_id/guest_id, option_id, created_at	One response per actor per moment; moment/time index
friendships	id, user_a, user_b, status	Unique unordered pair
response_visibility	user_id, moment_id, visibility	Unique pair
moment_approvals	moment_id, approver_id, decision, created_at	Approval history
share_cards	id, moment_id, user_id/guest_id, asset_key, created_at	Unique per (moment, actor)
reports	id, reporter_id, target_type, target_id, status	Status/target indexes
moderation_actions	id, actor_id, target_type, target_id, action	Audit index
outbox_events	id, type, payload, status, next_attempt_at	Pending-event index
push_tokens	id, user_id, platform, token	Unique token
notification_preferences	user_id, live_now, trending, friend_activity	1:1 with users
notification_deliveries	user_id, event_key, status, sent_at	Unique dedupe key; daily-count index
user_daily_stats	user_id, date, moments_joined, majority_matches, unique_people	Unique (user_id, date)

Entity	Key fields	Constraints / indexes
data_requests	id, user_id, type, status, requested_at, completed_at	Type/status index — export & deletion (Section 25)
audit_logs	actor_id, action, resource, resource_id, created_at	Actor/resource/time indexes
Added in v1.1. share_cards (Section 13) and data_requests (Section 25) were not in the v1.0 model. Both are required by PRD Sections 7.6 and 9.3 respectively and are small, additive tables — no migration risk to existing entities.		

8 Response Write Consistency

```

POST /v1/moments/:momentId/responses
|
v
Authenticate actor
|
v
Validate: moment exists -> moment is LIVE -> server time within
         active window -> option belongs to moment ->
         actor allowed to participate -> rate limit
|
v
BEGIN
  INSERT responses
  INSERT outbox_events
COMMIT
|
v
Return authoritative result to caller
|
v
Worker reads outbox -> projection update
|
v
RealtimePublisher -> crowd.snapshot

```

Client-side button disabling is a UX nicety, not a correctness mechanism — it is never relied on to prevent duplicate responses. Uniqueness is enforced at the database boundary (one response per actor per moment), and retries reuse the same idempotency key, so a network retry after a timeout returns the existing result instead of creating a second row.

9 Realtime Protocol

Supabase Broadcast is the initial transport. Postgres Changes is deliberately not used for the high-fanout crowd stream — it ties realtime throughput directly to write-ahead-log replication, which does not hold up at 100K concurrent viewers on a single moment.

Item	Value
Channel	moment:{momentId}
Event	crowd.snapshot
Payload	Versioned aggregate only — never private response rows
Subscription	Active moment by default
Reconnect	Exponential backoff with jitter
Fallback	GET /v1/moments/:id/result polling
Ordering	Client accepts only a strictly higher snapshot version
<pre>{ "event": "crowd.snapshot", "momentId": "m_123", "version": 1827, "generatedAt": "2026-08-12T09:00:00Z", "totalResponses": 48231, "optionCounts": { "yes": 30215, "no": 18016 }, "joinedLastMinute": 423, "volumeState": "mature" }</pre>	

Snapshots are generated at a bounded cadence under load, and volumeState distinguishes a genuinely small crowd from a mature one — this is the mechanism behind the PRD requirement that no moment ever display a fabricated or misleadingly small number (Section 7.1.2). The system never sends one realtime message per individual response to every viewer; that pattern does not scale and is explicitly disallowed in Section 23.

10 Feed Architecture

The feed is server-ranked, cursor-paginated, and client-buffered.

```
GET /v1/feed?cursor=<opaque>&limit=10
```

Candidate order:

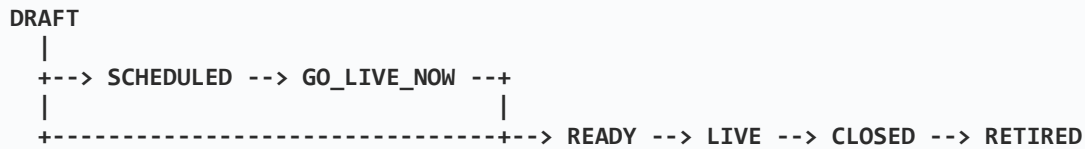
1. Live-now
2. Trending
3. Interest match
4. General/trending fallback

Filters: not expired -> eligible for audience -> not blocked/restricted
-> not recently seen by this actor

Client: renders current card, preloads next 3

No ML at MVP. The PRD calls for light personalization based on selected interest categories. Deterministic ranking is used instead of a model so behavior is explainable, cheap to operate, and testable — a hard requirement given the PRD's 500ms empty-state ceiling and the small initial editorial team producing 15–20 moments/day.

11 CMS and Editorial State



Moments tagged health, tragedy, or election-related require an explicit second-approver state before LIVE — the state machine enforces this rather than relying on editor discipline, which directly satisfies the PRD's acceptance criterion that no restricted-topic moment can go live without a second approver on record

12 Authentication and Privacy

- Registered identities are managed through Supabase Auth.
- Guest sessions are first-class actors for MVP participation — this is what lets a new install reach a first moment without a signup gate.
- Guest participation can be linked to a newly created user account, preserving response history on conversion.
- The backend derives identity from the authenticated session; it never trusts a client-supplied user ID.
- Response visibility is Public, Friends-only, or Private, set as an account default and overridable per moment.
- Privacy checks run before returning friend presence or friend response data — never as a client-side filter.
- Account deletion triggers a controlled workflow covering owned data, push tokens, friend relationships, and queued work.
- Contact sync is opt-in and revocable; the raw address book is not retained as a permanent application dataset.

Guest-to-Account Soft Prompt

PRD Section 7.2.1 allows guest play with “a soft prompt to save progress after 3 moments,” but doesn't specify where that count lives. It is tracked server-side, not client-side, so the prompt fires consistently across app reinstalls and multiple devices within the same guest session's lifetime.

- `guest_sessions.moments_responded_count` increments transactionally in the same write as each response insert (Section 8) — it cannot drift from the actual response count.
- `POST /v1/moments/:id/responses` returns a `promptAccountCreation` flag once the count reaches 3; the client owns when and how to surface the prompt, never whether the threshold was met.
- Dismissing the prompt does not reset the counter or suppress it permanently — the flag simply continues to be true on subsequent responses, and the client decides its own re-prompt cadence.
- On conversion, `guest_sessions.converted_user_id` links the session and its full response history to the new account; no response data is duplicated or re-written.

13 Sharing and Media Generation

PRD Section 7.6 requires a branded, shareable result card generated in two taps, with native OS share-sheet integration and a deep link back into the specific moment. This was not modeled in the v1.0 architecture and is added here as its own module rather than folded into responses/, since it has a distinct dependency (image rendering) and a distinct failure mode (a slow render should never block the response flow that produced the result being shared).

Step	Behavior
Trigger	User taps Share on a result screen (post- response only, never pre- response)
Render	API composes prompt, the actor's answer, and the current crowd result into a branded image; result is written to share_cards and Supabase Storage
Caching	Card is generated once per (moment, actor) and reused on repeat taps, not regenerated
Delivery	Signed asset URL returned to client; client hands it to the native OS share sheet
Deep link	Recipient's link resolves to the moment's live or closed- result view — no account required to view

Step	Behavior
Design constraint. Card generation is synchronous but bounded — target under 1 second — because it sits in a 2-tap flow with an 8%+ share-to-install conversion target riding on it (PRD Section 5). If render latency threatens that budget, generation moves to a queued job with a short client-side wait state rather than degrading the whole response endpoint.	

14 Notifications

PRD Section 7.3 specifies three notification types, a default cap of 3/day, independent per-type opt-out, and a hard rule that nothing is sent for an expired or closed moment. The v1.0 architecture listed the push component but not the enforcement mechanism; it is specified here.

Type	Trigger	Cap / guard
Live Now	Event-tied moment goes LIVE	Subject to daily cap; suppressed if moment closes before delivery
Trending Pulse	Poll crosses a rapid-response-volume threshold	Subject to daily cap; per-type opt-out
Friend Activity	A friend responds to a moment the actor hasn't seen	Subject to daily cap; per-type opt-out; respects friend's visibility setting

The daily cap and expiry check are enforced in the notification-delivery worker immediately before dispatch, using `notification_deliveries` as the count/dedupe source of truth — not at creation time, since a notification created hours before send can go stale by the time its slot comes up. Tapping any notification deep-links directly into the specific moment card.

15 Security Controls

Control	Requirement
Authentication	Provider-issued session; server validates identity
Authorization	Backend resource policy; role/ownership/visibility checks
Input validation	Strict DTO/schema validation on every API
Rate limiting	IP/device/user/guest-aware controls on expensive endpoints
Idempotency	Response, share-card, and notification side effects use dedupe keys
Secrets	Environment/secret manager only; never committed to the repo
Uploads	Signed upload path; size/type validation; private bucket
Admin	Separate role/permission checks; audit log
Logs	No tokens, passwords, or private response payloads in logs
Data deletion	User-initiated deletion and export workflows (see Section 25)

16 Performance Design

Target	Implementation
< 100ms swipe transition	3-card prefetch, local rendering, stable list virtualization
< 2s p95 counter update	Fast response write + aggregate snapshot + broadcast
< 3s cold start	Minimal startup dependencies; first feed payload prioritized
< 500ms normal feed loading	Prefetch, buffer stays ahead of the user
95% of viewers within 2s	Measured end-to-end across response -> snapshot -> delivery
< 1s share-card generation	Synchronous render with a queued fallback path (Section 13)

A swipe is never blocked on analytics, notification state, or nonessential profile data.

17 Failure Handling

Failure	Behavior
Realtime unavailable	Retain last result, switch to bounded polling, reconnect later
Response request timeout	Retry with same idempotency key; check authoritative result first
Duplicate response	Return existing authoritative response / conflict semantics
DB unavailable	Fail the write safely; never claim a response succeeded
Redis unavailable	Continue from DB where safe; rebuild projections
Worker failure	Retry with backoff; dead-letter and alert after threshold
Push provider failure	Retry; record final delivery failure
Storage failure	Retry upload; never persist unusable asset metadata

18 Observability

System	Metrics
Mobile	Cold start, feed render, swipe latency, crash-free sessions, realtime reconnects
API	RPS, p50/p95/p99 latency, 4xx/5xx rate, saturation
DB	Connections, CPU, slow queries, locks, transaction failures
Realtime	Connections, joins, message rate, delivery lag, disconnects
Workers	Queue age, retry count, dead-letter count, job latency
Product	DMJ, D1/D7/D30, live participation rate, share rate, session length

Every request and domain event carries a request/event ID, so a single response can be traced end-to-end through API submission, persistence, projection, and realtime delivery.

19 Testing Strategy

Layer	Tests
Unit	Moment state transitions, feed scoring, privacy rules, notification eligibility/cap, recap calculations
API	Auth, authorization, validation, errors, idempotency, pagination
Integration	DB constraints, transaction + outbox, worker retries, storage, realtime adapter
E2E	Install -> first moment, response -> result, friend presence, notification deep link, share flow, CMS publish
Load	1K/10K/50K/100K concurrent connections; hot-moment fanout
Chaos/failure	Realtime outage, DB outage, queue delay, reconnect storm, duplicate delivery

20 Repository Layout

```

playbyte/
  apps/
    mobile/
    admin/
  backend/
    src/
      modules/
      infrastructure/
      common/
    tests/
  supabase/
    migrations/
    seed/
  packages/
    contracts/
    api-client/
    config/
  docs/
    architecture.md    realtime.md    database.md    adr/
  load-tests/
  .github/workflows/

```

21 Scaling Plan

Stage	Architecture
MVP / early usage	Managed API + Supabase Postgres/Auth/Storage + Supabase Broadcast; minimal cache
Growing usage	Multiple stateless API instances + Redis/Valkey + separate worker capacity
High realtime load	Higher realtime capacity/plan; stricter snapshot cadence and subscription discipline
100K target	Formal capacity test, provisioned limits, headroom, incident runbook

Stage	Architecture
1M+ path	Evaluate dedicated realtime provider (e.g. AWS AppSync); read replicas; regional deployment; specialized event infrastructure if required

The application contract stays the same while infrastructure changes underneath the adapters — this is the payoff of keeping vendor calls behind interfaces from Section 5.

22 Forward Compatibility — V1.1 / V2 Readiness

This architecture is scoped to the MVP, but the PRD roadmap (Section 6) commits to specific V1.1 and V2 features. None of them are built now; this section records where each one attaches to the current design so V1.1 planning starts from an informed baseline rather than a surprise.

V1.1 / V2 feature
Comments & reactions
Groups / private moments
Opt-in duels
Sponsored moments
Pro tier / subscription billing
Creator tools

23 Compliance and Data Regulation

PRD Section 9.3 requires GDPR/CCPA-consistent handling, an age gate with no behavioral ad targeting below the regulated threshold, and opt-in/revocable contact sync. The v1.0 architecture stated the deletion and export requirement but did not model the underlying workflow; it is specified here.

Requirement	Implementation
Data export	User-initiated request creates a data_requests row; async worker assembles a portable export and delivers a signed, time-limited link
Data deletion	Same data_requests flow; deletion workflow from Section 12 executes owned data, push tokens, friendships, and queued work
Age gate	Enforced at signup on the auth module; unverified-age accounts are excluded from ad-targeting eligibility flags used by sponsored-moment insertion (Section 24)
Contact sync consent	Stored as a revocable consent record, not inferred from possession of synced data; revocation triggers deletion of the raw address book
Regional variance	Consent and retention rules are read from a policy table keyed by region, not hard-coded, since GDPR/CCPA and future regimes diverge

24 Non-Negotiable Engineering Rules

1. PostgreSQL is the source of truth.

2. Do not perform privileged business writes directly from the mobile client.
3. Do not use Postgres Changes for the high-fanout moment crowd stream.
4. Do not broadcast raw response events to every viewer.
5. Use compact, versioned crowd snapshots.
6. Response creation is idempotent and protected by a database uniqueness constraint.
7. Realtime subscriptions are limited to active moments by default.
8. The client falls back to polling when realtime is unavailable.
9. All authorization is enforced server-side.
10. Every asynchronous side effect is retry-safe.
11. Any new infrastructure service requires an architecture decision record.
12. Do not introduce a second database until a concrete workload justifies it.
13. Load test the actual Playbyte traffic model before declaring capacity.

25 Architecture Decisions and Revisit Conditions

Decision	Current choice	Revisit when
Backend topology	Modular monolith	Independent service scaling or ownership is justified
Primary DB	Supabase PostgreSQL	Measured storage/read/write limits or cost require a change
Realtime	Supabase Broadcast	Capacity, cost, SLA, or regional needs require a dedicated service
Cache	Redis/Valkey only when needed	DB load or latency requires it
Queue	Managed queue abstraction	Event throughput/replay needs justify a stream platform
Hosting	Managed containers	Operational scale/security needs justify AWS/ECS

26 Final Recommendation

Use Supabase as the starting platform. Keep PostgreSQL, Auth and Storage on Supabase. Use Supabase Broadcast for the first realtime implementation, but hide it behind a Playbyte realtime interface. Keep all product and authorization logic in the NestJS modular monolith. Add Redis and queue capacity based on measured load. When realtime scale or economics justify it, replace only that layer — potentially with AppSync, Ably, or a custom service.